

Project Report: Trekking Management Platform (TMA)

Student Details

- **Name:** Deepak Kumar Maurya
- **Roll Number:** 23f3004133
- **Email:** 23f3004133@ds.study.iitm.ac.in
- **Bio:** I am a student pursuing the Online BS Degree in Data Science and Applications at IIT Madras, passionate about building responsive, secure, and performant full-stack systems.

Project Details

- **Project Title:** Trekking Management Application
- **Description:** This project implements a full-stack **Trekking Management Platform (TMA)** designed to streamline the administration of adventure treks, automated staff assignments, and slot-based user booking operations. It addresses the challenges of trekking agencies by coordinating three distinct actors: **Admins** (who manage treks, staff, and overall platforms), **Staff Guides** (who lead treks and track participants), and **Trekkers** (who search, wishlist, review, and book treks).
- **Approch:** The app was built using Flask as the backend framework with a modular structure. It allow to trekkers to book the trek, keep wishlist, get notification about booking/cancellation, to staff to manage the trek status and to admin to manage all activity on plateform

AI/LLM Declaration

- **Extent of Use:** I have used ChatGPT (GPT-5.5) to assist me in writing frontend template's bootstrap and the flow of vue part. The extent of AI/LLM usage is around 15–20% for frontend and also I took help form it to deciding the codeflow of all three Async Operations.
All AI-assisted designs, All final implementation logic, debugging, and integration were manually validated to ensure correctness and alignment with projects's constraints.

Technologies Used

Technology / Library	Purpose & Rationale
Flask (v3.1.3)	Micro web framework selected for lightweight and flexible RESTful API development.
Vue 2 (v2.7.16)	Component-based progressive JS framework for building a high-speed, reactive single-page frontend.
SQLAlchemy / Flask-SQLAlchemy	Object-Relational Mapper (ORM) used to map database tables to Python classes securely, preventing SQL injections.
SQLite (via LocalConfig)	Embedded serverless database utilized for structured relational storage during local testing and development.
Flask-JWT-Extended (v4.7.4)	Enables stateless authentication using encrypted JWT tokens stored in the browser's local storage.
Flask-Caching (v2.4.0) & Redis	Caches high-traffic endpoints (such as public trek listings) to optimize latency, falling back automatically to in-memory caching if Redis is unavailable.
Flask-Mail (v0.10.0)	Integrates SMTP servers (e.g., Gmail) to dispatch automated daily reminders and monthly reports.
Python Daemon Threads (threading)	Custom asynchronous queue and scheduler built into the Flask lifecycle, eliminating heavy external worker daemons (like Celery/Celery Beat) while retaining async capabilities.
Bootstrap (v5.3.8)	Responsive styling framework ensuring accessibility across mobile and desktop devices.
Chart.js (v4.4.3)	Rich visualization library used on dashboards to depict monthly revenue, booking quantities, and trek difficulty distributions.

DB Schema Design & ER Diagram

Database Schema Details

1. `users` Table

- Columns:** `id` (INTEGER, PK, Autoincrement), `name` (VARCHAR(100), NOT NULL), `username` (VARCHAR(80), UNIQUE, NOT NULL, Index), `email` (VARCHAR(100), UNIQUE, NOT NULL, Index), `password` (VARCHAR(256), NOT NULL - Argon2/PBKDF2 salted hash), `role` (VARCHAR(20), NOT NULL, default 'user' - Values: 'admin', 'staff', 'user'), `phone` (VARCHAR(20)), `bio` (TEXT), `avatar_url` (VARCHAR(300)), `status` (VARCHAR(20), NOT NULL, default 'active' - Values: 'active', 'blacklisted', 'inactive'), `created_at` (DATETIME, default UTC Now), `updated_at` (DATETIME, default UTC Now)
- Constraints / Rationale:** Unique indexes on `username` and `email` prevent double-registrations. Indexing lookup attributes optimizes login speeds.

2. `trekker_info` Table (1-to-1 extension of `users` for Trekkers)

- Columns:** `id` (INTEGER, PK, Autoincrement), `user_id` (INTEGER, FK `users.id`, UNIQUE, NOT NULL), `experience_level` (VARCHAR(20), default 'Beginner' - 'Beginner', 'Intermediate', 'Expert'), `fitness_level` (VARCHAR(20), default 'Medium' - 'Low', 'Medium', 'High'), `preferred_difficulty` (VARCHAR(20)), `emergency_contact` (VARCHAR(100)), `emergency_phone` (VARCHAR(20)), `medical_notes` (TEXT)
- Rationale:** Kept separate from the core `users` table to maintain database normalization and avoid storing empty/null columns for `admin` or `staff` accounts.

3. `staff_info` Table (1-to-1 extension of `users` for Staff/Guides)

- Columns:** `id` (INTEGER, PK, Autoincrement), `user_id` (INTEGER, FK `users.id`, UNIQUE, NOT NULL), `specialization` (VARCHAR(100) - e.g., 'High Altitude', 'Winter'), `certification` (TEXT), `years_experience` (INTEGER, default 0), `language` (VARCHAR(200))
- Rationale:** Encapsulates qualifications specific only to guides, enabling custom assignments based on difficulty levels.

4. `treks` Table

- Columns:** `id` (INTEGER, PK, Autoincrement), `name` (VARCHAR(200), NOT NULL), `location` (VARCHAR(200), NOT NULL, Index), `difficulty` (VARCHAR(20), NOT NULL, Index - 'Easy', 'Moderate', 'Hard'), `duration` (INTEGER, NOT NULL - in days), `total_slots` (INTEGER, NOT NULL), `available_slots` (INTEGER, NOT NULL), `staff_id` (INTEGER, FK `users.id`, Nullable - Guide assigned), `status` (VARCHAR(20), default 'Pending', Index - 'Pending', 'Approved', 'Open', 'Closed', 'Completed'), `start_date / end_date` (DATE), `description / highlights` (TEXT - Pipe-separated lists), `included / not_included` (TEXT), `image_url` (VARCHAR(300)), `altitude` (INTEGER), `price` (FLOAT, default 0.0), `meeting_point` (VARCHAR(300)), `equipment_needed` (TEXT)

5. `bookings` Table (Many-to-Many junction between `users` and `treks`)

- Columns:** `id` (INTEGER, PK), `user_id` (INTEGER, FK `users.id`, NOT NULL), `trek_id` (INTEGER, FK `treks.id`, NOT NULL), `booking_date` (DATETIME, default UTC Now), `status` (VARCHAR(20), default 'Booked' - 'Booked', 'Cancelled', 'Completed'), `payment_status` (VARCHAR(20), default 'Pending' - 'Pending', 'Paid', 'Refunded'), `payment_method` (VARCHAR(30)), `transaction_id` (VARCHAR(50)), `amount` (FLOAT, default 0.0), `notes` (TEXT)

6. `reviews` Table (Many-to-Many junction)

- Columns:** `id` (INTEGER, PK), `user_id` (INTEGER, FK `users.id`, NOT NULL), `trek_id` (INTEGER, FK `treks.id`, NOT NULL), `rating` (INTEGER, NOT NULL - Range: 1-5), `comment` (TEXT)
- Constraints:** Unique Constraint `uq_user_trek_review` on (`user_id`, `trek_id`) prevents a trekker from writing multiple reviews for the same trek.

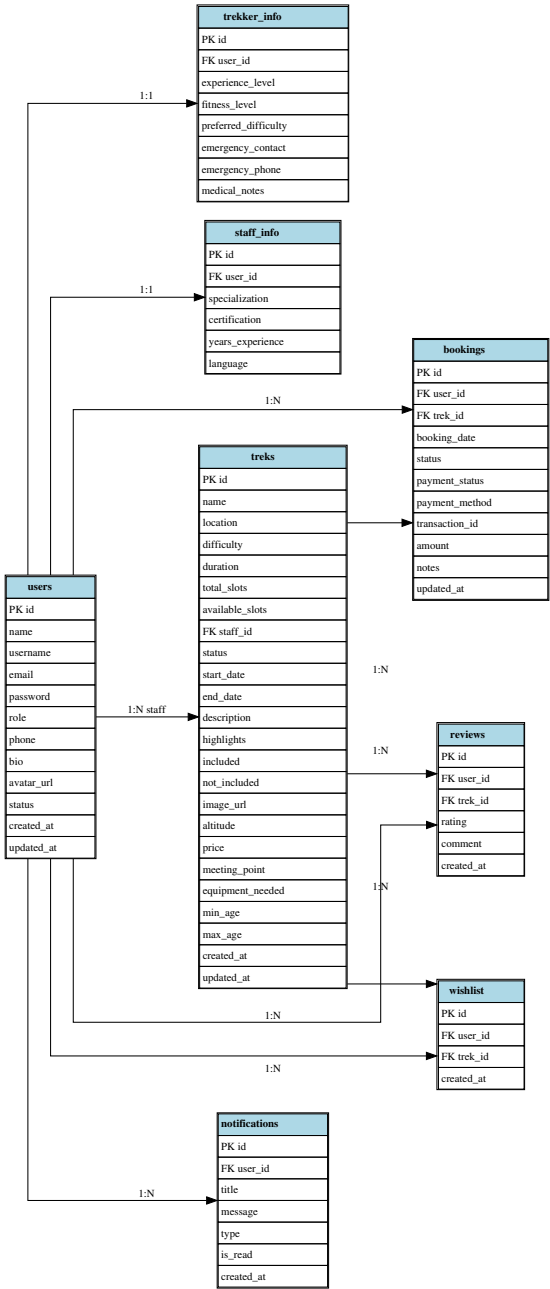
7. `wishlist` Table (Many-to-Many junction)

- Columns:** `id` (PK), `user_id` (FK), `trek_id` (FK). Unique Constraint `uq_wishlist` prevents duplicates.

8. `notifications` Table

- Columns:** `id` (PK), `user_id` (FK), `title` (VARCHAR), `message` (TEXT), `type` (VARCHAR), `is_read` (BOOLEAN).
-

Entity-Relationship (ER) Diagram



API Design & Endpoints

All secure endpoints require validation via the `Authorization: Bearer <JWT_TOKEN>` HTTP header.

1. Authentication Endpoints (`/api/auth`)

- `GET /check?field=username&value=XYZ` — Check availability of username/email during registration.
- `POST /register` — Register a new Trekker.
- `POST /login` — Log in and receive a JWT.
- `GET /profile` — Retrieve logged-in user profile, including specialized `trekker_info`/`staff_info` fields.
- `PUT /profile` — Modify profile details.

2. Admin Endpoints (`/api/admin`)

- `GET /dashboard` — Retrieve dashboard KPIs (revenue, slots, trek distribution, popular treks).
- `GET /stats` — Fetch analytics for Chart.js (monthly booking counts and revenue graphs).
- `GET /search?q=query` — System-wide search of Treks, Users, and Staff.
- `GET/POST /treks` — Retrieve list of treks or insert a new trek.
- `PUT/DELETE /treks/<id>` — Modify/Delete specific trek details (deletion allowed only if there are no active bookings).

- `POST /treks/<id>/assign-staff` — Allocate a specific Staff Guide to a trek.
- `GET/POST /staff` — List staff or create a new Staff account.
- `PUT/DELETE /staff/<id>` — Modify/Delete a Staff member.
- `GET /users` — List all registered trekkers.
- `PUT /users/<id>/status` — Block/activate user accounts (blacklisted users cannot log in).
- `GET /bookings` — List all bookings on the platform.
- `DELETE /reviews/<id>` — Moderation endpoint to remove user reviews.

3. Trekker (User) Endpoints (`/api/trekker`)

- `GET /dashboard` — Fetch user-specific stats (bookings counter, wishlist items, unread alerts).
- `GET /treks` — Search/filter available treks (cached for performance).
- `GET /treks/<id>` — Fetch a single trek's details.
- `POST /bookings` — Purchase slots for a trek (automatically deducts slot count and runs age-limit verification).
- `PUT /bookings/<id>/cancel` — Cancel a booking (refunds slots back to the trek capacity).
- `POST /reviews` — Post a trek review.
- `GET/POST/DELETE /wishlist` — Retrieve, add, or delete wishlist entries.
- `GET/PUT /notifications` — List or mark in-app alerts as read.
- `POST /export-bookings` — Triggers the asynchronous worker thread to compile the user's booking history into a CSV.
- `GET /export-bookings/status/<task_id>` — Monitor export status.

4. Staff Endpoints (`/api/staff`)

- `GET /dashboard` — Fetch details on treks led and total participants guided.
- `GET /treks` — List treks assigned to this staff member.
- `PUT /treks/<id>` — Advance trek status (e.g., set status to `Started` or `Completed`).
- `GET /treks/<id>/participants` — Retrieve information of trekkers registered for the guide's trek.

Architecture & Features

```
trekking-management-application/
├── backend/
│   ├── models/
│   │   ├── models.py          # SQLAlchemy Models & Serialization
│   │   └── routes/
│   │       ├── auth.py        # Login, Registration & Profile modification
│   │       ├── admin.py       # Admin dashboards and CRUDs
│   │       ├── staff.py       # Guide dashboards & status progression
│   │       └── user.py        # Trekker booking, wishlist, & reviews
│   ├── app.py                 # Application Initialization & Setup
│   ├── config.py              # System configuration (caching, mail, DB path)
│   ├── extensions.py          # Flask Extensions definitions
│   ├── celery_app.py          # Celery configuration & Beat scheduling setup
│   ├── tasks.py               # Celery async and scheduled task definitions
│   └── trigger.py             # Script to manually run scheduled tasks instantly
├── frontend/
│   └── src/
│       ├── views/             # Vue templates for dashboards, login, tables
│       ├── router.js          # Client-side routing and role guards
│       ├── App.vue            # Main template entry
│       ├── style.css          # Core CSS styling rules
│       ├── main.js            # Vue app initialization
│       ├── vite.config.js     # Port settings and API reverse proxy
│       ├── package.json       # Frontend dependency declarations
│       ├── requirements.txt    # Backend dependency declarations
│       └── README.md          # System running guidelines
```

Key Architectural Highlights

- **Celery Worker & Beat Scheduler:** Background tasks are processed using **Celery** with Redis as the message broker. **Celery Beat** runs on a schedule to trigger daily email reminders and monthly activity reports, while the worker executes these along with on-demand CSV exports.
- **Role-Based Guards:** The frontend router protects views with role-based checks. Unauthorized transitions default to the user's corresponding dashboard, protecting route access.
- **Database Constraints & Slot Validation:** Slot reservations occur within SQL transactions. If slot capacity is exceeded or other validation checks fail, the transaction is rolled back to prevent database anomalies.

Video

- **Presentation Video Link:** [Presentation Video](#)